

第11讲

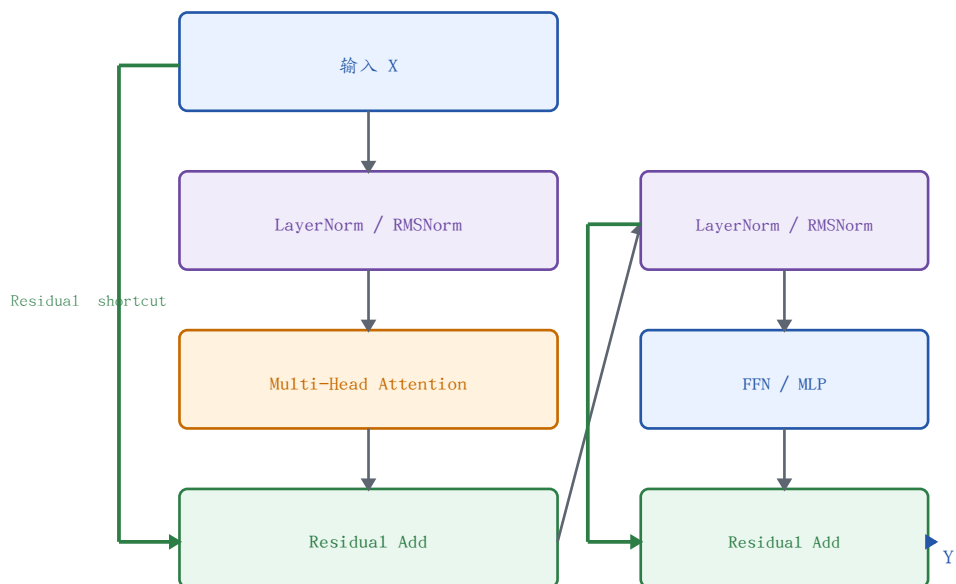
一个完整 Transformer Block 到底怎样工作？

把 Attention、Residual、LayerNorm、FFN 第一次完整拼起来

本讲目标

学完后，你应该能从输入 X 开始，逐步解释一个 Transformer Block 内部发生什么；理解 Residual、LayerNorm、FFN 各自解决什么问题；区分 Attention 与 FFN 的职责；看懂 Pre-Norm / Post-Norm；并能追踪主要 Shape。

一个 Transformer Block 的整体结构（先看全局）



现代 LLM 常见 Pre-Norm: 先归一化, 再 Attention/FFN, 然后与残差流相加

先记住一句话

Attention 负责 token 之间交换信息；FFN 负责每个 token 自己做非线性加工；Residual 保留主干；Norm 稳定数值。

1. 为什么现在终于可以拼一个完整 Block?

前面十讲我们分别学了 Token、Embedding、神经网络、训练、Attention、Multi-Head Attention 和位置编码。现在已经有足够零件，把 Transformer 的“基本发动机”拼出来。

- 输入 token 已经变成 hidden states;
- Attention 已经会让 token 之间交换信息;
- 但只有 Attention 还不够：深层网络还要稳定训练、保存旧信息、做更强的非线性变换。

本讲核心

Transformer Block

不是一个新的神秘算法，而是把几种我们已经见过的神经网络结构以固定套路组合起来。

2. Transformer Block 的四个主角

组件	最通俗的职责	你可以先怎么记
Multi-Head Attention	让 token 彼此读取相关信息	交流 / 路由
Residual Connection	保留原信息，同时叠加新变化	高速公路 / shortcut
LayerNorm / RMSNorm	把数值尺度拉回稳定范围	稳压器
FFN / MLP	每个 token 单独做非线性加工	本地加工厂

不要混淆

Attention 与 FFN 都会改变向量，但只有 Attention 显式把不同 token 的信息混合在一起；标准 FFN 通常逐 token 独立处理。

3. 第一步：Block 的输入到底是什么？

设序列长度为 n ，模型隐藏维度为 d_{model} 。进入一个 Block 的输入通常可以写成：

```
X in  $\mathbb{R}^{(n \times d_{\text{model}})}$ 
```

每一行对应一个 token 当前的 hidden state。这里已经不是原始 Token ID，也不是最初初始的 Embedding；在深层网络中，它是前一层不断加工后的“当前表示”。

关键直觉

一个 token 在第 1 层、第 10 层、第 30 层的向量可以完全不同，因为它已经不断吸收上下文并被多次重写。

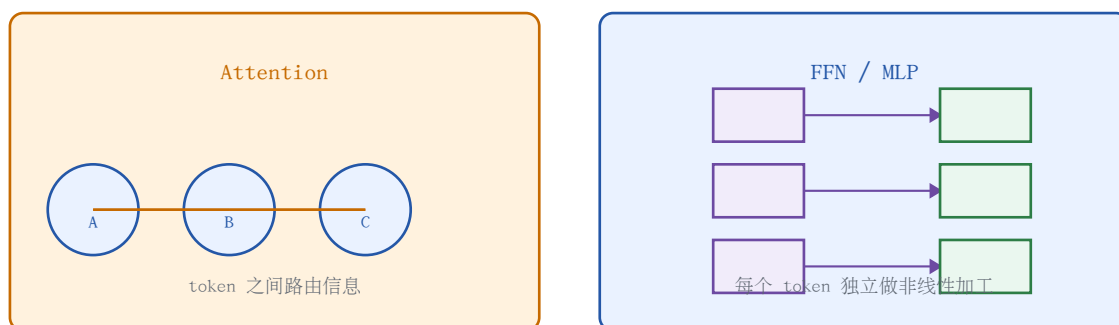
4. Attention 子层：先让 token 之间交流

在一个 Block 里，Attention 的作用可以概括为：当前 token 根据 Q/K 计算“应该关注谁”，再按权重读取 V。

```
Attention(X) -> [n, d_model]
```

注意：Multi-Head Attention 内部会拆成多个 head，但合并并经过 W_O 后，输出通常重新回到 d_{model} 。这样后面才能继续进入 residual stream。

Attention 与 FFN 的分工：先“交流”，再“各自加工”



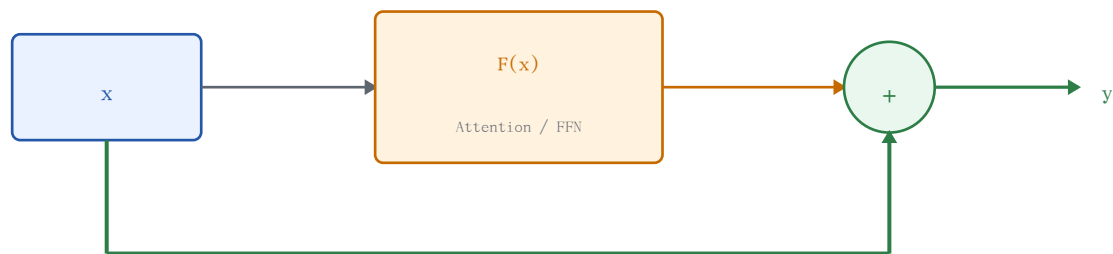
这是理解 Transformer Block 的一个非常实用的二分法，但真实网络功能会更复杂。

Shape 直觉

主干通常保持 $[n, d_{model}]$ 。Attention 内部可以拆头，但出来后需要回到同一主维度。

5. 为什么还要 Residual Connection?

Residual Connection: 让信息拥有“主干公路 + 加工支路”



shortcut: 原始 x 直接保留下来

数学: $y = x + F(x)$ 。网络只需学习“在原信息上应该补什么变化”。

如果每一层都把输入完全“推翻重算”，深层网络会很难训练。Residual Connection 让网络可以在原表示上学习一个“增量修改”：

$$y = x + F(x)$$

通俗比喻

不是每一层都重写整篇论文，而是在原稿上加批注、补信息、做修订。这样旧信息有一条直接通路继续向后流。

6. Residual 为什么对深层网络训练重要？

从优化角度看，Residual 提供了一条更直接的信息和梯度通路。即使某个子层暂时学得不好，主干 x 仍可以绕过它继续传递。

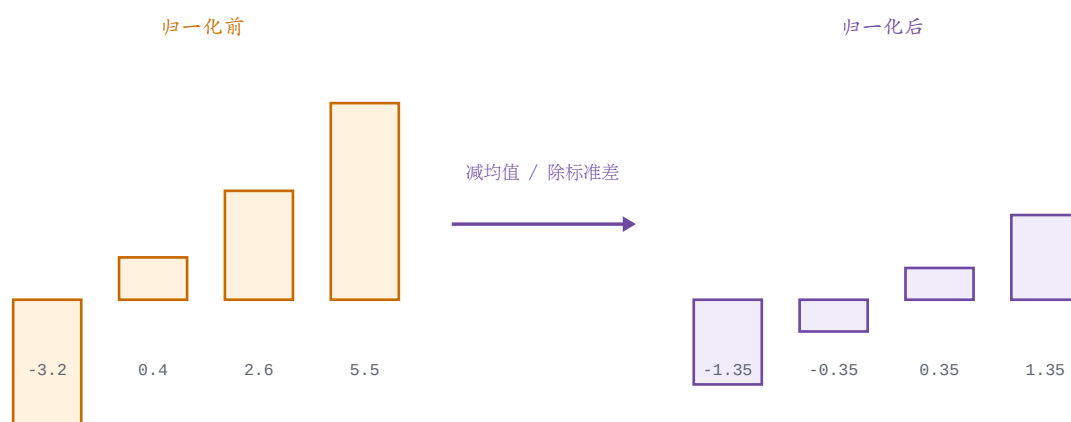
- 前向传播：旧表示不会被子层完全覆盖；
- 反向传播：梯度可以沿 shortcut 更直接地向前层传播；
- 学习任务：子层只需要学习“需要补上的变化”，而不是从头构造完整表示。

不要误解

Residual 并不意味着旧信息永远原封不动。经过许多层连续相加，表示仍然可以发生巨大变化。

7. LayerNorm 到底在“Norm”什么？

LayerNorm：对一个 token 的特征维度做归一化



目的不是把信息“洗掉”，而是把特征尺度拉到更稳定的数值范围，再用可学习 γ/β 调整

LayerNorm 对一个 token 的特征维度做归一化。简化公式：

$$\hat{x} = (x - \text{mean}(x)) / \sqrt{\text{var}(x) + \text{eps}} \quad y = \gamma * \hat{x} + \beta$$

其中 γ 和 β 是可学习参数。Norm 的目标不是把所有 token 变成一样，而是让数值尺度更稳定，帮助深层网络训练。

初学者最容易错

LayerNorm 不是“把每个 token 变成均值 0 方差 1 后就结束”。后面还有可学习的缩放 γ 和平移 β 。

8. LayerNorm 为什么不是 BatchNorm?

对比	LayerNorm	BatchNorm
主要归一化维度	单个样本 / token 的特征维	跨 batch 的统计量
是否依赖 batch 组成	弱 / 不依赖同批样本统计	强
序列模型中常见度	非常常见	相对少
推理时是否需要运行均值	通常不需要	经典 BatchNorm 需要训练统计

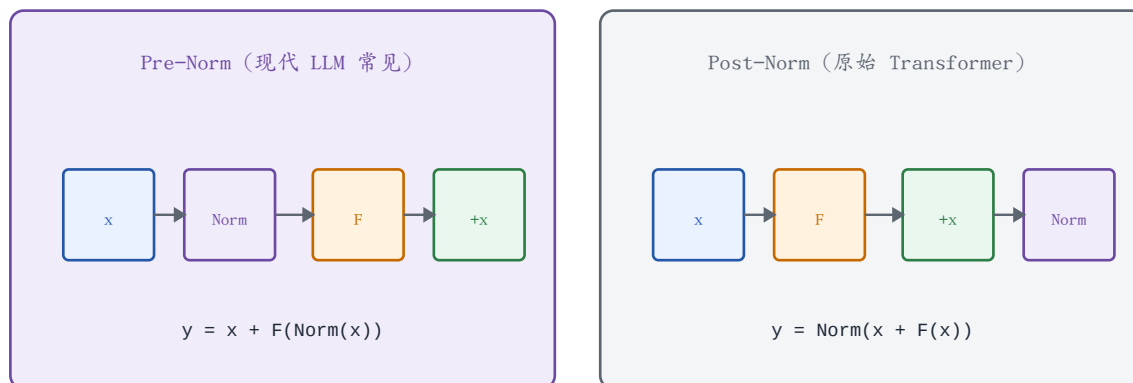
Transformer 处理变长序列、小 batch、在线生成时，LayerNorm 这种“不依赖其他样本”的特性更自然。

现代 LLM 提醒

很多现代 LLM 使用 RMSNorm，它省略“减均值”这一步，只按均方根尺度归一化。后面看到 RMSNorm 不要以为出现了完全不同的模型。

9. Pre-Norm 与 Post-Norm: Norm 应该放哪里?

Pre-Norm 与 Post-Norm: LayerNorm 放在哪里?



两种结构都是真实存在的。初学时不要把“LayerNorm 必须在某一边”当成绝对规则。

原始 Transformer 采用 Post-Norm 风格；很多现代大语言模型更常采用 Pre-Norm 或其变体，因为深层训练往往更稳定。

现在不用站队

你只需要会读结构：先 Norm 再 F 再 Add，还是先 F + Add 再 Norm。论文和代码里必须看清楚。

10. FFN / MLP 到底是什么？

FFN / MLP：每个 token 单独做“升维加工 → 激活 → 降维”



重要：标准 FFN 不让 token 彼此通信；同一组权重逐 token 处理。跨 token 信息主要由 Attention 负责。

经典 Transformer FFN 可以写成：

$$\text{FFN}(x) = W2 * \text{activation}(W1*x + b1) + b2$$

它通常先把 d_{model} 升到更大的 d_{ff} ，再做非线性激活，然后投影回 d_{model} 。经典 Transformer 常用 ReLU；后来的 LLM 常见 GELU、SiLU、SwiGLU 等。

一句话

Attention 让 token “互相说话”；FFN 让每个 token “自己消化刚听到的信息”。

11. FFN 为什么要先升维再降维？

`d_model` 是 `residual stream` 的主干宽度，但在 FFN 内部，模型可以暂时进入一个更大的特征空间，让非线性组合更丰富。

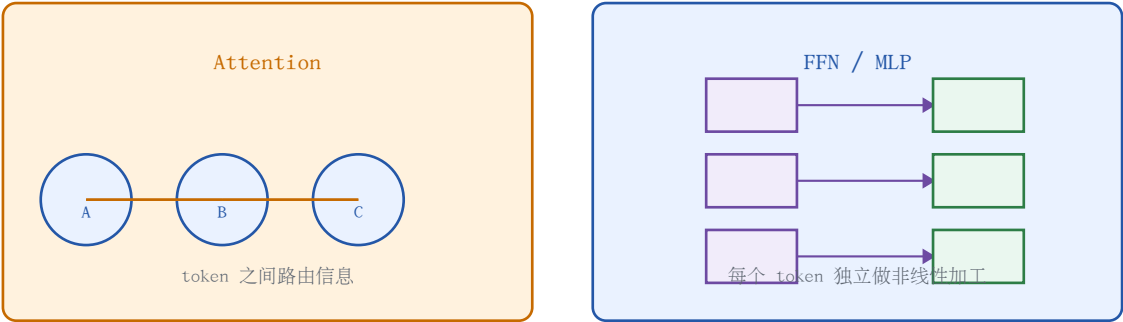
阶段	Shape（典型）	含义
输入	<code>[n, d_model]</code>	主干表示
W1 后	<code>[n, d_ff]</code>	扩展到更宽特征空间
激活后	<code>[n, d_ff]</code>	非线性筛选 / 组合
W2 后	<code>[n, d_model]</code>	回到主干宽度

为什么必须回 `d_model`？

因为随后通常还要与 `residual stream` 相加。两条支路维度需要能一一对应。

12. Attention 与 FFN：一个 Block 里到底谁负责什么？

Attention 与 FFN 的分工：先“交流”，再“各自加工”



这是理解 Transformer Block 的一个非常有用的二分法，但真实网络功能会更复杂。

问题	Attention	FFN
会不会混合不同 token 的信息？	会	标准 FFN 通常不会
主要计算	Q/K/V + 加权读取	线性层 + 非线性激活
主要 Shape 变化	内部拆头，输出回 d_model	d_model -> d_ff -> d_model
最有用的直觉	通信 / 信息路由	逐 token 特征加工

科研提醒

这是理解功能的“第一层近似”，不是严格解释。真实网络中 Attention 和 MLP 的功能分工会更复杂，也会随层深和训练而变化。

13. 把前半块 Block 逐步走一遍 (Pre-Norm 版本)

假设输入是 X 。第一半通常可以写成:

$$A = X + \text{Attention}(\text{Norm}(X))$$

- Step 1: $\text{Norm}(X)$, 先把数值尺度调整稳定;
- Step 2: 用规范化后的表示计算 Q/K/V 和 Multi-Head Attention;
- Step 3: 得到 Attention 输出;
- Step 4: 与原始 X 相加, 得到 A 。

注意残差从哪里来

Residual 加的是子层之前的主干 X , 而不是 $\text{Norm}(X)$ 。代码阅读时这是一个容易忽略的小细节。

14. 再走后半块 Block (Pre-Norm 版本)

第二半可以写成:

$$Y = A + \text{FFN}(\text{Norm}(A))$$

- Step 5: $\text{Norm}(A)$;
- Step 6: 经过 FFN: 升维 $\rightarrow$ 激活 $\rightarrow$ 降维;
- Step 7: FFN 输出与 A 相加;
- Step 8: 得到这个 Block 的输出 Y 。

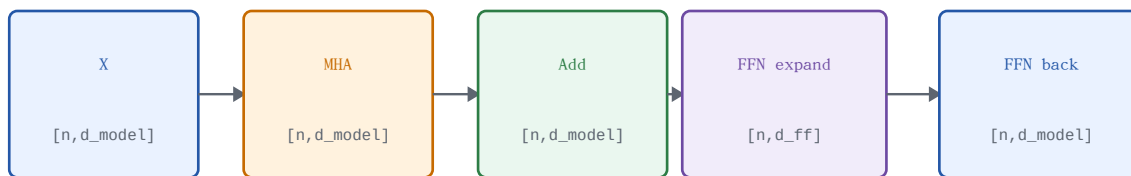
完整最小公式

$$A = X + \text{Attn}(\text{Norm}(X))$$

$$Y = A + \text{FFN}(\text{Norm}(A))$$

15. Shape 从头到尾怎么走？

Shape 最重要的直觉：一个 Block 前后主维度通常保持 $[n, d_{\text{model}}]$



FFN 内部可以临时变宽，但回到 residual stream 时需要回到 d_{model} ，才能做相加。

这也是为什么 Residual Add 要求两条支路的 shape 可以对应相加。

虽然 Attention 和 FFN 内部都会临时改变结构，但一个标准 Block 的输入和输出通常都保持主干 shape:

```
[batch, n, d_model] -> [batch, n, d_model]
```

这件事很重要

正因为每层输入输出主维度一致，才能像搭积木一样重复堆叠几十层甚至更多。

16. 一个 Block 里到底有多少可学习参数？

忽略 bias 和 Norm 小参数，最主要的参数来自 Attention 投影和 FFN:

Attention: W_Q, W_K, W_V, W_O FFN: W_1, W_2 (或门控变体中的更多矩阵)

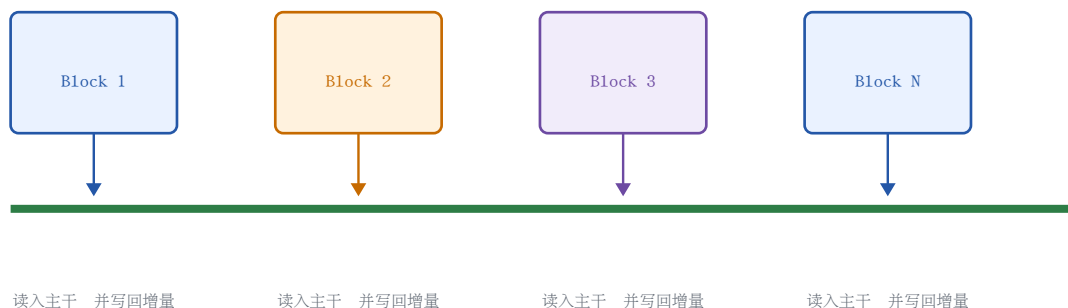
如果粗略按经典结构估算: Attention 约是 $4 \cdot d_{\text{model}}^2$ 量级; FFN 如果 d_{ff} 大约是 $4 \cdot d_{\text{model}}$, 则两层 FFN 参数约 $8 \cdot d_{\text{model}}^2$ 。

一个重要直觉

在很多经典 Transformer 配置中, FFN 参数量往往比 Attention 还大。所以 “Transformer = Attention” 是过度简化。

17. Residual Stream: 理解深层 Transformer 的更好心智模型

Residual Stream: 把深层 Transformer 想成一条持续流动的信息主干



这是一个有用的理解框架：每层 Attention/FFN 都在“读取并修改”同一维度的表示流。

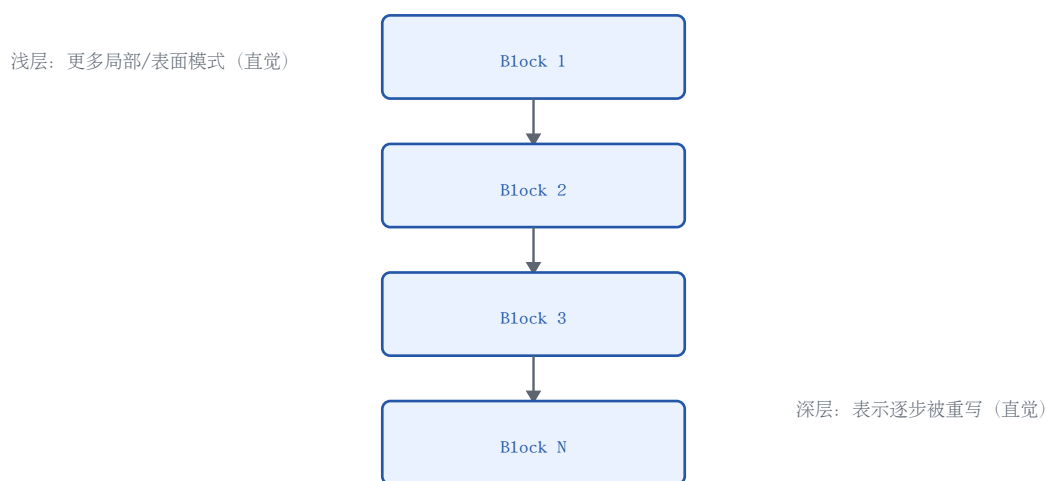
把 d_{model} 维的 hidden state 想成一条贯穿全网的“残差流”。每个 Attention/FFN 子层从这条流里读信息，计算一个更新量，再把更新写回去。

为什么这个模型有用

它能帮助你理解：深层网络不是每层完全重新编码，而是在同一条主干表示上连续叠加许多更新。

18. Transformer 为什么要堆很多 Block?

Transformer = 同类 Block 一层层堆叠，而不是只运行一次



不要机械断言“某一层固定负责某种知识”；层功能是训练后涌现出的分工。

一个 Block 只能完成一次“跨 token 路由 + 局部非线性加工”。堆很多层后，信息可以被反复重组、组合和抽象。

- 浅层可以先形成较基础的局部模式；
- 后续层在已经上下文化的表示上继续加工；
- 更深层可以组合更长的依赖和更抽象的特征。

不要过度机械化

“第 3 层一定学语法、第 20 层一定学事实”这种说法通常过于绝对。真实层功能会依模型、任务和训练而变化。

19. Decoder-only LLM 的 Block 有什么特殊之处?

GPT 类模型使用 decoder-only Transformer。它的 Self-Attention 需要 Causal

Mask: 当前位置不能读取未来 token。

```
X -> Norm -> Causal Multi-Head Self-Attention -> Add -> Norm -> FFN -> Add
```

与最基础的 Encoder Block 相比, 核心区别之一就是注意力可见范围。经典完整 Transformer Decoder 还包含 Cross-Attention, 用于读取 Encoder 输出; GPT 这种 decoder-only 模型通常没有 Encoder-Cross-Attention 这一层。

和上一讲连接

位置编码告诉模型“位置在哪里”; Causal Mask 规定“哪些位置允许看”。二者会同时进入 decoder-only Block。

20. 原始 Transformer 与现代 LLM Block 有哪些常见变化？

部件	原始/经典直觉	现代 LLM 常见变化
Norm	LayerNorm	Pre-Norm、RMSNorm
Activation	ReLU	GELU、SiLU、SwiGLU
Attention	MHA	GQA / MQA 等高效变体
Position	Sinusoidal	RoPE 等
FFN	两层 MLP	门控 MLP (如 SwiGLU)

最重要的认识

“Transformer Block” 是一类架构骨架，不是永远固定的唯一配方。论文里要看具体实现，而不是只看到 Transformer 三个字就默认所有细节一样。

21. 训练时和推理时，Block 有什么不同？

Block 的数学结构基本相同，但运行方式不同。训练时通常一次并行处理整段序列，并保存中间激活用于反向传播；自回归推理时则不断追加新 token，并尽量复用以前的 K/V。

阶段	训练	自回归推理
序列处理	整段并行	逐 token 生成
是否反向传播	需要	通常不需要
中间激活	大量保存	较少
KV Cache	不是核心必需	非常重要

后面会专门讲

KV Cache、Prefill、Decode、Latency 和 Throughput 会在推理系统部分详细解释。

22. 为什么 Transformer Block 能不断变深而不立即崩掉？

没有单一原因。它依赖多种结构共同作用：Residual 让信息/梯度有捷径；Norm 控制数值尺度；Attention 和 FFN 提供不同形式的表示变换；合理初始化、优化器和训练策略保证参数更新稳定。

不要寻找一个“唯一英雄”

Transformer 的成功不是因为 Attention
单独解决了一切，而是整个架构、硬件并行、数据和优化共同构成系统。

23. 常见误区集中纠正

误区	正确理解
“Transformer Block 就是 Attention”	错误；FFN、Residual、Norm 同样关键
“Residual 什么都不做，只是复制”	错误；它决定信息与梯度如何跨层流动
“LayerNorm 会让所有 token 一样”	错误；它只规范数值尺度，且有可学习参数
“FFN 是 token 之间通信”	标准 FFN 通常逐 token 独立处理
“每层输入输出必须完全相同”	主 shape 通常相同，但数值表示持续变化
“现代 LLM 都和 2017 Transformer 一模一样”	错误；Norm、激活、位置、Attention 变体均有演化

24. Agent Systems 连接：别把“模型内部 Block”与“Agent 模块”混在一起

Transformer Block 是 LLM 内部的神经网络层。Agent 的 Memory、Tool、Planner、Environment、Message Passing 是模型外部/系统级机制。

层级	Transformer Block	Agent System
粒度	神经网络内部张量计算	一个或多个模型 + 工具 + 状态 + 环境
信息载体	hidden state / tensor	文本、结构化消息、memory、tool result
交互方式	矩阵运算	调用、通信、规划、行动循环
研究问题	表示与计算机制	系统行为、可靠性、协作、成本、反馈

对 AI Systems Science 很重要

未来研究 Agent Systems 时，经常要区分“底层模型机制导致的问题”和“系统编排导致的问题”。Transformer Block 属于前者的基础。

25. 小实验：自己跑一个极小 Transformer Block

材料包中的 Python 代码只依赖 NumPy，手写一个极小的 Pre-Norm Transformer Block: LayerNorm → 2-head causal self-attention → residual → LayerNorm → FFN → residual。

运行文件

04_tiny_transformer_block_demo.py

程序会打印：输入 shape、Attention 权重、第一条 Residual 前后数值、FFN 中间 shape，以及最终输出。你可以把 d_model、head 数、序列长度改小/改大，观察 shape 如何变化。

26. 练习题

- 1) 一个 Transformer Block 的四个核心组成是什么？
- 2) Residual Connection 为什么写成 $y = x + F(x)$ ？它解决了什么问题？
- 3) LayerNorm 的主要目的是什么？它和 BatchNorm 的直觉区别是什么？
- 4) Pre-Norm 与 Post-Norm 的结构差异是什么？
- 5) FFN 为什么可以理解为“逐 token 本地加工”？
- 6) 为什么 FFN 常把 d_{model} 暂时升到更大的 d_{ff} ？
- 7) Attention 与 FFN 在一个 Block 中的功能分工是什么？
- 8) 为什么一个 Block 的输入输出通常都保持 $[n, d_{model}]$ ？
- 9) Decoder-only LLM 的 Block 为什么需要 Causal Mask？
- 10) 为什么“Transformer = Attention”是错误的过度简化？

27. 练习答案

- 1) Multi-Head Attention、Residual Connection、Norm (LayerNorm/RMSNorm) 和 FFN/MLP。
- 2) 让原信息沿 shortcut 直接保留，子层只学习增量 $F(x)$ ，同时改善深层网络的信息和梯度传递。
- 3) 稳定单个 token 特征维度的数值尺度；LayerNorm 不依赖同 batch 其他样本的统计量，而 BatchNorm 依赖跨 batch 统计。
- 4) Pre-Norm: $y=x+F(\text{Norm}(x))$; Post-Norm: $y=\text{Norm}(x+F(x))$ 。
- 5) 标准 FFN 对每个位置使用相同权重独立计算，不显式读取其他 token。
- 6) 在更宽特征空间中进行更丰富的非线性组合，再投影回 residual stream 的 d_{model} 。
- 7) Attention 主要负责 token 间信息路由；FFN 主要负责每个 token 的非线性特征加工。
- 8) 这样才能做 Residual Add，并让多个 Block 按同一接口稳定堆叠。
- 9) 自回归训练/生成时当前位置不能读取未来 token，否则会泄露答案。
- 10) Transformer 的能力来自 Attention、FFN、Residual、Norm、位置机制、训练与优化共同作用，Attention 只是核心部件之一。

28. 本讲总结：你应该能把整个 Block 画出来

$X \rightarrow \text{Norm} \rightarrow \text{Multi-Head Self-Attention} \rightarrow + X \text{ (Residual)} \rightarrow A \rightarrow \text{Norm} \rightarrow \text{FFN / MLP} \rightarrow + A \text{ (Residual)} \rightarrow Y$

最低闭环能力

看到一个 Transformer Block 图时，至少能逐个问：Norm 在哪边？Attention 是否 causal？Residual 从哪里绕过？FFN 如何升降维？输出如何回到 `d_model`？

下一讲

第12讲进入 GPT 到底是什么结构？：Encoder、Decoder、Encoder-Decoder 的区别，以及为什么现代大语言模型大量采用 Decoder-only。

参考原始工作：Vaswani et al., Attention Is All You Need (2017); He et al., Deep Residual Learning for Image Recognition (2015/2016); Ba et al., Layer Normalization (2016)。